

# max\_element 정리

범위 안에서 가장 큰 원소의 위치를 찾아 반복자로 반환하는 C++ STL 알고리즘입니다.

## 사용처 (Where)

- 배열이나 vector의 최댓값을 찾을 때
- 최댓값의 위치(인덱스)가 필요할 때
- 사용자 정의 기준으로 가장 큰 원소를 찾을 때

## 방법 (How to Use)

- 탐색할 [first, last) 범위 지정
- max\_element(first, last) 호출
- 반환된 반복자를 last와 비교
- \*it로 값, distance()로 위치 확인

## 기본 정보

- 필요 헤더: #include <algorithm>
- 반환형: 입력 범위의 반복자
- 기본 문법: max\_element(v.begin(), v.end())
- 같은 최댓값이 여러 개면 첫 번째 위치 반환

## 왜 써야 할까? (Why)

- 직접 반복문을 쓰는 코드보다 짧고 명확함
- 값과 위치를 모두 쉽게 얻을 수 있음
- 배열, vector 등 반복자 범위에 공통 사용

## 주요 함수

| 함수/문법                    | 의미                        |
|--------------------------|---------------------------|
| max_element(first, last) | [first, last) 범위의 최댓값 반복자 |
| *it                      | 반환된 위치의 값                 |
| it - v.begin()           | vector에서 인덱스 계산           |
| distance(first, it)      | 일반 반복자의 거리 계산             |
| max_element(..., comp)   | 비교 기준을 직접 지정              |

## 기본 코드 예시

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v{4, 9, 2, 9, 5};
    auto it = max_element(v.begin(), v.end());
    cout << *it << '\n';
    cout << it - v.begin() << '\n';
}
```

# max\_element 비교와 응용

직접 탐색과 비교하고, 값과 위치를 안전하게 얻는 방법을 확인합니다.

## 시간 복잡도

| 동작                       | 복잡도  |
|--------------------------|------|
| max_element(first, last) | O(n) |
| 반복자 역참조 (*it)            | O(1) |
| vector 인덱스 계산            | O(1) |

## STL vs 직접 구현 vs C 스타일

| 구분    | STL max_element | 직접 반복문      | 정렬 후 확인      |
|-------|-----------------|-------------|--------------|
| 표현력   | 의도가 명확          | 비교 로직 직접 작성 | 필요 이상의 작업    |
| 코드 길이 | 짧고 표준적          | 초기화와 갱신 필요  | 정렬 코드 필요     |
| 시간    | O(n)            | O(n)        | O(n log n)   |
| 추천 상황 | 최댓값/위치 탐색       | 특수 규칙이 많을 때 | 전체 정렬도 필요할 때 |

## 응용: 가장 높은 점수와 번호 출력하기

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> score{72, 88, 95, 81};
    auto it = max_element(score.begin(), score.end());
    cout << "score: " << *it << '\n';
    cout << "index: " << it - score.begin() << '\n';
}
```

## 처음 배울 때 자주 하는 실수

- 빈 범위에서 반환된 end() 반복자를 역참조함
- 함수가 최댓값 자체를 반환한다고 생각함
- 두 번째 인수 last가 범위에 포함된다고 생각함
- 같은 최댓값 중 마지막 위치가 반환된다고 생각함

## 비슷한 항목과 차이

- min\_element는 범위의 가장 작은 원소 위치를 반환
- minmax\_element는 최솟값과 최댓값의 위치를 한 번에 반환
- max는 보통 두 값 중 큰 값 자체를 반환
- sort는 전체 범위의 순서를 바꾸지만 max\_element는 바꾸지 않음

한 줄 정리: max\_element는 '범위의 최댓값과 그 위치가 필요할 때' 선택하는 STL입니다.

## 사용법 숙지를 위한 기본 예제

### 기본 예제 1. 배열의 최댓값 찾기

배열도 begin(), end() 대신 포인터 범위로 전달할 수 있습니다.

```
int a[] = {3, 7, 2, 6};
auto it = max_element(a, a + 4);
cout << *it << '\n';
```

실행 결과: 7

### 기본 예제 2. 음수에서 최댓값 찾기

모두 음수여도 가장 큰 값을 올바르게 찾습니다.

```
vector<int> v{-8, -2, -10};
auto it = max_element(v.begin(), v.end());
cout << *it << '\n';
```

실행 결과: -2